Minimum Discarded Spans

Delete as few complete ID pairs as possible so that the remaining execution log is properly nested.

Problem statement

You are given an event array of even length. Every distinct span ID occurs **exactly twice**: the first appearance starts that span, and the second closes it. A valid log behaves like a stack: spans may be disjoint or completely nested, but two spans cannot cross.

You may discard a span only by deleting **both** occurrences of its ID. Find the **minimum number of spans** to discard to make the remaining event sequence valid.

Input: event count `n`, followed by `n` IDs (spaces or newlines). **Output:** one integer. **Limits:** 2 <= n <= 2000; n even; IDs up to 10^9.

One worked test case

INPUT	OUTPUT
6 1 2 1 3 2 3	1

Why? Spans 1 and 2 cross, and spans 2 and 3 cross. Discarding span 2 leaves `[1,1,3,3]`, which is valid. One discard is optimal.

Idea to solve

- Convert each ID into an interval from its first to its second appearance. We want to **keep the maximum number of non-crossing intervals**; answer = total spans - maximum kept.
- Let `dp[l][r]` be the maximum number of spans that can be kept using only events from indices `l` through `r`.
- For a right endpoint `r`, either skip the span ending there, or keep its matching pair `(k,r)`. Keeping it splits the other kept spans into `[l,k-1]` and `[k+1,r-1]`.

> **Transition:** `dp[l][r] = max(dp[l][r-1], 1 + dp[l][k-1] + dp[k+1][r-1])` when the pair at `r` starts at `k >= l`. Treat empty intervals as zero. Otherwise only the skip case applies. **Complexity:** O(n^2) time and O(n^2) memory.

Similar problems to practise

AtCoder ABC217 F - Make Pair - Closest structural practice: non-crossing pairings using interval DP (different objective).

LeetCode 312 - Burst Balloons - Practice splitting a range into two independent DP subintervals.

CSES - Nested Ranges Check - Practice understanding nested and containing intervals.

PROBLEM 2 / 03 | MST + TREE DP

Secure Network Resilience

First construct the cheapest network; then maximize the number of nodes saved from a spreading failure.

Problem statement

There are **N** data clusters and one virtual central hub (node 0). You may connect cluster `i` directly to the hub for `hub_costs[i]`, or use any of **M** bidirectional fiber links `(u,v,cost)`. Build the **unique minimum-cost connected network**; it is a tree.

The hub fails at time 0; infection then spreads one edge per second. Before each spread, you may quarantine one currently healthy cluster. Quarantining removes that cluster from the network, so its descendants can be protected. A node counts as **saved** only if it is neither infected nor quarantined. Find the maximum number saved.

Guarantee: The hub has at most two neighbors in the optimal tree; every other node has at most three neighbors (so each rooted node has at most two children). **Limits:** N up to 10^6, M up to 10^7; costs up to 10^9.

One worked test case

INPUT

```
1
5 8
60 78 63 6 107
2 3 57
1 3 30
2 4 64
3 4 4
5 1 56
2 5 71
1 4 45
3 5 50
```

OUTPUT

```
4
```

Why? The cheapest tree has edges `0-4, 4-3, 3-1, 3-2, 3-5`. At time 1, quarantine node 4: only that node is lost; nodes 1, 2, 3, and 5 are saved.

Idea to solve

- **Part A - Minimum spanning tree:** Add a virtual node 0, connect it to every cluster `i` with cost `hub_costs[i]`, and add all fiber links. Run Kruskal with DSU to obtain the minimum-cost tree.
- **Part B - Rooted tree DP:** Root the MST at 0. Compute subtree sizes bottom-up. Let `save[u]` be the most nodes that can be saved below u when u is infected.
- For two children `a,b`, quarantine one child immediately and let the failure enter the other: take the better of `size[a]-1 + save[b]` and `size[b]-1 + save[a]`.

> **Base cases:** Leaf: 0. One child a: `size[a]-1`. **Answer:** `save[0]`. **Complexity:** Kruskal O((N+M) log(N+M)); tree DP O(N). Use iterative traversal to avoid deep recursion.

Statement caveats: The written input format includes T, but samples omit it; the case above uses T=1. Also, this sample MST gives node 3 four neighbors, apparently contradicting the stated degree-at-most-3 guarantee. The binary-tree DP relies on that guarantee for general tests.

Similar problems to practise

Codeforces 1689C - Infected Tree - Very close to the infection/quarantine tree DP stage.

LeetCode 1168 - Optimize Water Distribution - Same trick: virtual hub node + minimum spanning tree.

Advertiser-Publisher Ad Campaign

Find the strongest bottleneck path between two nodes while obeying a strict hop limit.

Problem statement

You have an **undirected, weighted graph** with **N** nodes and **M** edges. An edge weight is its transfer capacity. For any path from source S to destination D, its impact is the **minimum edge capacity** on that path.

Find a path using at most **K edges** that maximizes this minimum capacity. If no valid path exists, print **0**.

Input: `N M K`, then `S D`, followed by M lines `u v w`. **Output:** one integer. **Limits:** N, M, K up to 10^5; positive integer edge capacities.

One worked test case

INPUT

```
5 6 3
1 5
1 2 10
1 3 50
2 4 40
3 4 20
4 5 30
2 3 60
```

OUTPUT

```
20
```

Why? Path `1-3-4-5` uses 3 edges and has bottleneck `min(50,20,30)=20`. Path `1-2-4-5` has bottleneck 10. Higher-bottleneck routes require more than 3 edges.

Idea to solve

- **Binary search the answer:** Suppose we want an impact of at least X. We may use only edges with capacity `w >=` X.
- **Check X with BFS:** On the filtered unweighted graph, run BFS from S. BFS finds the minimum number of edges required to reach each node. The target is feasible exactly when `dist[D] <=` K.
- **Use monotonicity:** If threshold X is possible, every smaller threshold is possible. If X is impossible, every larger threshold is impossible. Keep the greatest feasible X.

> **Complexity:** Each BFS is O(N+M), and there are O(log W) thresholds where W is the largest edge weight. Total O((N+M) log W) time and O(N+M) memory. **Watch out:** K counts edges, not intermediate stops; output 0 if D cannot be reached within K edges.

Similar problems to practise

LeetCode 1102 - Path With Maximum Minimum Value - Same maximin / bottleneck-path objective, on a grid and without a hop limit.

LeetCode 787 - Cheapest Flights Within K Stops - Practise hop-constrained routes; note that K stops means K+1 edges here.

The linked problems train related techniques; none is claimed to be an exact copy of the corresponding question.